Introdução

> A otimização prematura é a raiz de todos os males.
>
> Donald Knuth

Este livro destina-se a programadores que já sabem*C está* em um nível suficiente. Por quê? É improvável que você conheça apenas algumas linguagens interpretadas como Perl ou*Python* , algumas pessoas vão querer aprender imediatamente.*montador* . Usando*Sim* e*Ao combinar diferentes linguagens de montagem* , utilizando cada uma para fins específicos, é possível alcançar resultados muito bons. Além disso, os programadores*A linguagem C* já possui algum conhecimento sobre a arquitetura do processador, as características da computação de máquina, a organização da memória e outros aspectos que não são tão fáceis para um iniciante entender. Portanto, é necessário estudar mais a fundo.*montador* depois*C é* definitivamente mais fácil do que outras linguagens de alto nível.*C* é o conceito de "*O programador precisa gerenciar a alocação de memória no heap, incluindo ponteiros* , por exemplo. Todo esse conhecimento será útil no estudo de linguagem assembly, ajudando a obter uma visão mais completa da arquitetura e a ter uma compreensão mais abrangente da linguagem.*uma ideia* de como seus programas são executados em*Sim* . Mas esse conhecimento requer aprofundamento e estruturação.

Quero enfatizar que a leitura deste livro não exige nenhum conhecimento de Linux (exceto, é claro, o conhecimento de "como criar um ambiente Linux").*arquivo de texto* "e "como executar um programa no console"). E, em geral, a única coisa que expressa a orientação para o Linux é o usado*sintaxe* de montagem e*ABI* . Programadores de linguagem assembly em*DOS* e*O Windows* é usado.*Sintaxe* Intel , mas em sistemas *nix é comum usar*Sintaxe* AT&T . As partes em linguagem assembly do kernel Linux são escritas em sintaxe AT&T.*compilador O GCC* gera listagens de código assembly e assim por diante.

A maior parte das informações deste livro pode ser usada para programação não apenas em *nix, mas também em*No Windows* , você só precisa esclarecer alguns recursos dependentes do sistema (por exemplo,*ABI*).

Vale a pena?

Ao escrever código assembly, você deve sempre estar ciente se um determinado trecho de código realmente deve ser escrito em assembly. É importante ponderar os prós e os contras; compiladores modernos podem otimizar o código e alcançar desempenho comparável (ou até melhor, se a versão inicial em assembly do programador for inadequada).

A principal desvantagem da linguagem assembly é a futura impossibilidade de portabilidade do programa resultante para outras plataformas.

x86 ou IA-32?

Você provavelmente já ouviu o termo "*arquitetura x86* . Em geral, é bastante vago, e aqui está o porquê. O próprio nome*x86* ou*80x86* vem do princípio pelo qual a Intel nomeou seus processadores:

- Intel 8086 - 16 bits;
- Intel 80186 - 16 bits;
- Intel 80286 - 16 bits;
- Intel 80386 - 32 bits;
- Intel 80486 - 32 bits.

Esse*A lista* continua. A convenção de nomenclatura, em que cada geração de processadores recebia um nome terminado em 86, criou o termo "*x86* ". Mas se você olhar mais atentamente, poderá ver que o "processador"Tanto o antigo 8086 de 16 bits quanto o novo i7 podem ser chamados de *x86 . É por isso que as extensões de 32 bits eram chamadas de arquitetura.IA-32* (abreviação de Intel)*Arquitetura* de 32 bits). Obviamente, a capacidade de executar programas de 16 bits permanece e é usada com sucesso (e nem tanto) nas versões de 32 bits.*Windows* . Consideraremos apenas o modo de 32 bits.

```
.data /* coloque o seguinte no segmento de dados
                                                                    */

hello_str: /* nossa string */
        .string "Olá, mundo!\n"

                                /* comprimento da string */
        .set hello_str_length, . - hello_str - 1

.text /* coloque o seguinte no segmento de código */
```

```
.globl main /* main é um símbolo global, visível
                                fora do arquivo atual */
.type main, @function /* main é uma função (não dados) */


main:
        movl $4, %eax /* coloque o
                                número da chamada de sistema de escrita = 4 no registrador %eax */

        movl $1, %ebx /* primeiro parâmetro - no registrador %ebx; o
                                número do descritor de arquivo stdout
                                - 1 */

        movl $hello_str, %ecx /* segundo parâmetro - no registrador %ecx;
                                    um ponteiro para a string */

        movl $hello_str_length, %edx /* terceiro parâmetro - no
                                          registrador %edx; comprimento da string */

        int $0x80 /* chama a interrupção 0x80 */

        movl $1, %eax /* sai da chamada de sistema número -1 */
        movl $0, %ebx /* passa 0 como valor do parâmetro */
        int $0x80 /* chama exit(0) */

        .size main, . - main /* tamanho da função main */
```

Registros

Um registrador é uma pequena porção de memória de alta velocidade localizada no processador. Ele é usado para armazenar os resultados de cálculos intermediários, bem como algumas informações para controlar a operação do processador. Como os registradores estão localizados diretamente no processador,*O acesso* aos dados armazenados neles é muito mais rápido do que o acesso aos dados na RAM .

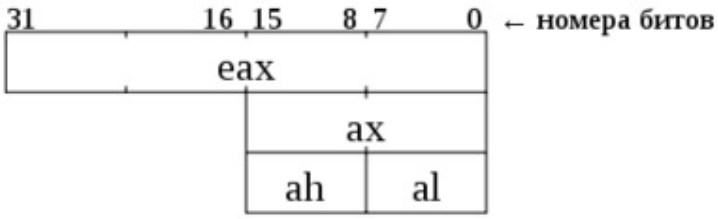
Todos os registradores podem ser divididos em dois grupos: de usuário e de sistema. Os registradores de usuário são usados ao escrever programas "regulares". Estes incluem *os registradores principais do programa* (EPRs).programa *básicoregistradores de execução* ; todos eles estão listados abaixo), bem como registradores do coprocessador matemático , registradores MMX ,*XMM* (SSE, SSE2, SSE3). Системные регистры (регистры управления, регистры управления памятью, регистры отладки, машинно-специфичные регистры *MSR* и другие) здесь не рассматриваются. Более подробно см.(Intel® 64 and IA-32 Architectures Software Developer's Manual, Volume 1: Basic Architecture, 3.2 Overview of the basic execution environment.

Регистры общего назначения (РОН, англ. *General* Purpose Registers, сокращённо *GPR*). Размер - 32 бита.

- `%eax` : Accumulator register - аккумулятор, применяется для хранения результатов промежуточных вычислений.
- `%ebx` : Base register - базовый регистр, применяется для хранения адреса (указателя) на некоторый объект в памяти.
- `%ecx` : Counter register - счетчик, его неявно используют некоторые команды для организации циклов (см. loop).
- `%edx` : Data register - регистр данных, используется для хранения результатов промежуточных вычислений и ввода-вывода.
- `%esp` : Stack pointer register - указатель стека. Содержит адрес вершины стека.
- `%ebp` : Base pointer register - указатель базы кадра стека (англ. stack frame). Предназначен для организации произвольного доступа к данным внутри стека.
- `%esi` : Source index register - индекс источника, в цепочечных операциях содержит указатель на текущий элемент-источник.
- `%edi` : Destination index register - индекс приёмника, в цепочечных операциях содержит указатель на текущий элемент-приёмник.

Эти регистры можно использовать "по частям". Например, к младшим 16 битам регистра `%eax` можно обратиться как `%ax` . А `%ax` , в свою *очередь*, содержит две однобайтовых половинки, которые могут

использоваться как самостоятельные регистры: старший `%ah` и младший `%al` . Аналогично можно обращаться к `%ebx` / `%bx` / `%bh` / `%bl` , `%ecx` / `%cx` / `%ch` / `%cl` , `%edx` / `%dx` / `%dh` / `%dl` , `%esi` / `%si` , `%edi` / `%di` .



Не следует бояться такого жёсткого закрепления назначения использования регистров. Большая их часть может использоваться для хранения совершенно произвольных данных. Единственный случай, когда нужно учитывать, в какой *регистр* помещать данные - использование неявно обращающихся к регистрам команд. Такое поведение всегда чётко документировано.

Сегментные регистры:

`%cs` : Code segment - описывает текущий сегмент кода.

`%ds` : Data segment - описывает текущий сегмент данных.

`%ss` : Stack segment - описывает текущий сегмент стека.

`%es` : Extra segment - дополнительный сегмент, используется неявно в строковых командах как сегмент-получатель.

`%fs` : F segment - дополнительный сегментный регистр без специального назначения.

`%gs` : G segment - дополнительный сегментный регистр без специального назначения.

В ОС Linux используется плоская модель памяти (*flat memory model*), в которой все *сегменты* описаны как использующие всё адресное *пространство* процессора и, как правило, явно не используются, а все адреса представлены в виде 32-битных смещений. В большинстве случаев программисту можно даже и не задумываться об их существовании, однако *операционная система* предоставляет специальные средства (*системный вызов* `modify_ldt()`), позволяющие описывать нестандартные *сегменты* и работать с ними. Однако такая потребность возникает редко, поэтому тут подробно не рассматривается.

Регистр флагов `eflags` и его младшие 16 бит, регистр `flags` . Содержит информацию о состоянии выполнения программы, о самом микропроцессоре, а также информацию, управляющую работой некоторых команд. *Регистр* флагов нужно рассматривать как *массив* битов, за каждым из которых закреплено определённое *значение*. *Регистр* флагов напрямую не доступен пользовательским программам; изменение некоторых битов `eflags` требует привилегий. Ниже перечислены наиболее важные флаги.

`cf` : carry flag, флаг переноса:

1 - во время арифметической операции был произведён перенос из старшего бита результата;

0 - переноса не было;

`zf` : zero flag, флаг нуля:

1 - результат последней операции нулевой;

0 - результат последней операции ненулевой;

`of` : overflow flag, флаг переполнения:

1 - во время арифметической операции произошёл перенос в/из старшего (знакового) бита результата;

0 - переноса не было;

`df` : direction flag, флаг направления. Указывает направление просмотра в строковых операциях:

1 - направление "назад", от старших адресов к младшим;

0 - направление "вперёд", от младших адресов к старшим.

Есть команды, которые устанавливают флаги согласно результатам своей работы: в основном это команды, которые что-то вычисляют или сравнивают. Есть команды, которые читают флаги и на основании флагов принимают решения. Есть команды, логика выполнения которых зависит от состояния флагов. В общем, через флаги между командами неявно передаётся дополнительная *информация*, которая не записывается непосредственно в результат вычислений.

Указатель команды eip (instruction pointer). Размер - 32 бита. Содержит *указатель* на следующую команду. *Регистр* напрямую недоступен, изменяется неявно командами условных и безусловных

переходов, вызова и возврата из подпрограмм.

Стек

Мы полагаем, что читатель имеет *опыт* программирования на *Си* и знаком со структурами данных типа *стек*. В микропроцессоре *стек* работает похожим образом: это область памяти, у которой определена *вершина* (на неё указывает `%esp`). Поместить новый элемент можно только на вершину стека, при этом новый элемент становится вершиной. Достать из стека можно только верхний элемент, при этом вершиной становится следующий элемент. У вас наверняка была в детстве игрушка-пирамидка, где нужно было разноцветные кольца надевать на общий стержень. Так вот, эта пирамидка - отличный пример стека. Также можно провести аналогию с составленными стопкой тарелками. На разных архитектурах *стек* может "расти" как в сторону младших адресов (принцип описан ниже, подходит для *x86*), так и старших.

```
Содержимое стека  Адреса в памяти

.                 .
.                 .
.                 .
+----------------+ 0x0000F040
|                |
+----------------+ 0x0000F044 <-- вершина стека (на неё указывает %esp)
|     данные     |
+----------------+ 0x0000F048
|     данные     |
+----------------+ 0x0000F04C
.                 .
.                 .
.                 .
+----------------+ 0x0000FFF8
|     данные     |
+----------------+ 0x0000FFFC
|     данные     |
+----------------+ 0x00010000 <-- дно стека
```

Стек растёт в сторону младших адресов. Это значит, что последний записанный в *стек* элемент будет расположен по адресу младше остальных элементов стека.

При помещении нового элемента в *стек* происходит следующее (принцип работы команды *push*):

значение %esp уменьшается на размер элемента в байтах (4 или 2);

новый элемент записывается по адресу, на который указывает `%esp`.

```
.                 .
.                 .
.                 .
+----------------+ 0x0000F040 <-- новая вершина стека (%esp)
|  новый элемент |
+----------------+ 0x0000F044 <-- старая вершина стека
|     данные     |
+----------------+ 0x0000F048
.                 .
.                 .
.                 .
+----------------+ 0x0000FFFC
|     данные     |
+----------------+ 0x00010000 <-- дно стека
```

При выталкивании элемента из стека эти действия совершаются в обратном порядке(принцип работы команды `pop`):

содержимое памяти по адресу, который записан в `%esp`, записывается в регистр;

а значение адреса в `%esp` увеличивается на размер элемента в байтах (4 или 2).

```
.                 .
.                 .
.                 .
+-----------------+ 0x0000F040 <-- старая вершина стека
| верхний элемент | -------------> записывается в регистр
+-----------------+ 0x0000F044 <-- новая вершина стека (%esp)
|      данные     |
+-----------------+ 0x0000F048
.                 .
.                 .
.                 .
+-----------------+ 0x0000FFFC
|      данные     |
+-----------------+ 0x00010000 <-- дно стека
```

Память

В *Cu* после вызова `malloc(3)` программе выделяется *блок памяти*, и к нему можно получить *доступ* при помощи указателя, содержащего *адрес* этого блока. В ассемблере то же самое: после того, как программе выделили *блок памяти*, появляется возможность использовать указывающий на неё *адрес* для всевозможных манипуляций. Наименьший по размеру элемент памяти, на который может указать *адрес*, - *байт*. Говорят, что *память* адресуется побайтово, или *гранулярность* адресации памяти - один *байт*. Отдельный *bit* pode ser especificado como*o endereço* do byte que contém isso*bit* , e o número desse bit no byte.

Há, no entanto, mais um detalhe a observar. O código do programa está localizado na memória, portanto, obtê-lo é um processo complexo.Também é possível enviar *um endereço .Pilha* também é*bloco de memória* , e o desenvolvedor pode obter*Um ponteiro* para qualquer elemento da pilha localizado abaixo do topo. É assim que eles se organizam.*Acesso* a elementos arbitrários da pilha.

Ordem dos bytes. Little-endian e big-endian.

Operacional*memória éMatriz* de valores binários, 0 e 1. Não abordaremos a ordem dos bits em um byte, pois é difícil especificá-la.*O endereço* de um único bit não é possível; apenas*o endereço* do byte que contém isso*Como os bytes* são organizados em uma palavra na memória? Vamos supor que temos o número `0x01020304` . Ele pode ser escrito como uma sequência de bytes:

começando pelo byte mais significativo: `0x01 0x02 0x03 0x04` - big-endian
começando pelo byte menos significativo: `0x04 0x03 0x02 0x01` - little-endian

Essa sequência de bytes está localizada na RAM.*o endereço* da palavra inteira na memória -*Endereço* do primeiro byte da sequência.

Se o mais novo for colocado primeiro*byte* (*A entrada* começa pela "extremidade menor") - esta é a ordem.*O formato de byte* é chamado de little-endian, ou "Intel". É o formato usado nos processadores.*x86* .

Se o mais velho for localizado primeiro*byte* (*A entrada* começa pela "extremidade maior") - esta é a ordem.*O formato de byte* é chamado de big-endian.

A ordem little-endian tem uma vantagem importante. Veja só*Escrevendo* o número `0x00000033` :

```
0x33 0x00 0x00 0x00
```

Se você ler isso como um valor de dois bytes*Se lermos o valor* 0x0033 , obtemos `0x33` . Se o lermos como um único byte, obtemos `0x33` . Esse truque também funciona na escrita. Claro, não podemos ler o número `0x11223344` como um valor.*byte* , porque obtemos `0x44` , o que está incorreto. Portanto, o número lido deve caber no alvo.*intervalo* de valores.

Veja também

- O artigo "Endianness" em en.wikipedia.org
- O artigo "Ordem de bytes" em ru.wikipedia.org

Atenção! Se você encontrar um erro em nosso site, selecione-o e pressione Ctrl+Enter.

© Universidade Nacional Aberta "INTUIT", 2026 | www.intuit.ru